Compiler design

1. Lexical analysis:
   - → Regular expression
   - → Rules for RE
   - → Conversion of RE to NFA
   - → Conversion of NFA with ε transition to without ε transition NFA
   - → NFA to DFA conversion
   - → DFA simulation

Lexical analyser doesn't consider '#include' lines, blank lines, spaces and comments.

eg. int x;
for (i=0; i<=10; i++)
{
    setvar x;
}

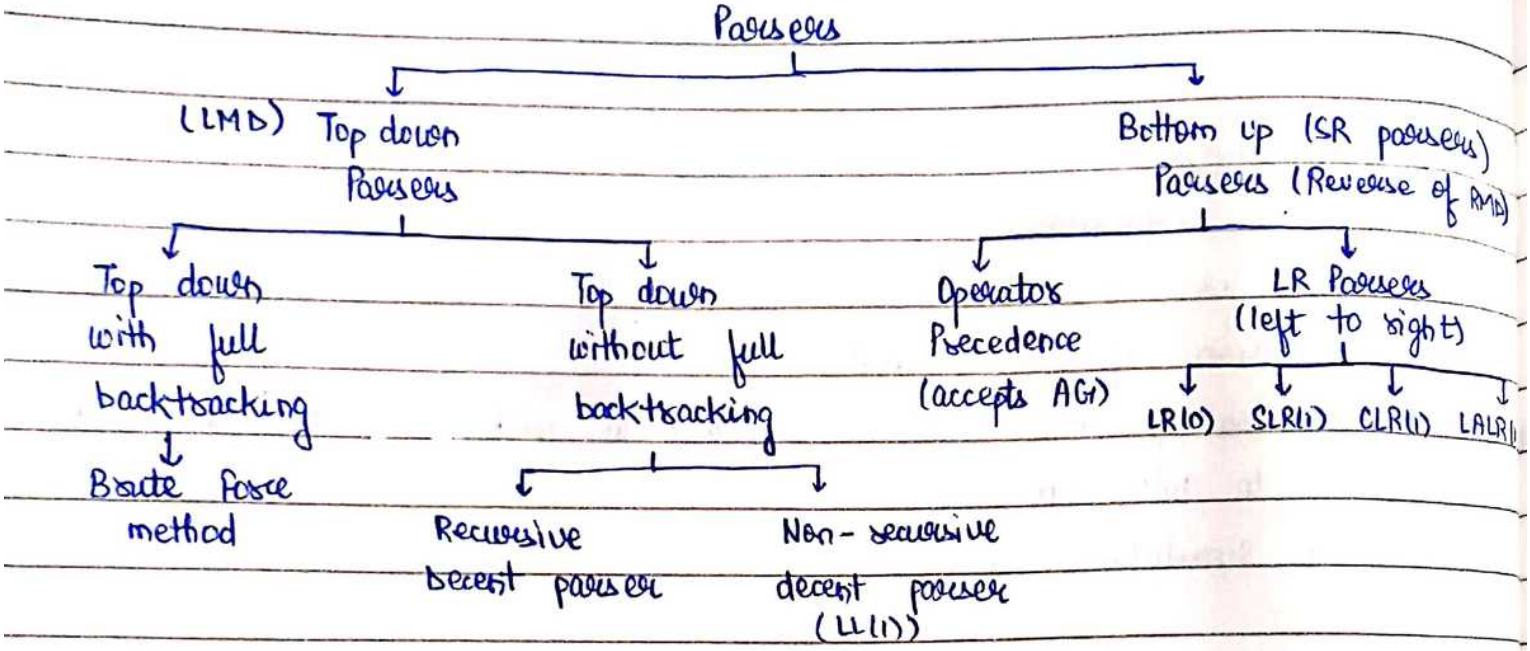
Total tokens = 21

eg. printf("The nos are %d %d", x, y); Total tokens = 9

scanf("%d", &x); Total tokens = 8

2. Syntax analysis:
   Remove unambiguous elements from grammar.
   Remove left recursion.

Scanned by CamScanner



Q. S → aABe w = abbcde

A → Abc / b

B → d

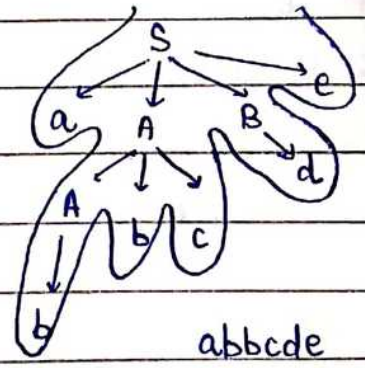
Sol.ⁿ LMD: Left Most Derivation

S → a**A**Be

S → a**A**bcBe

S → abbc**B**e

S → abbcde



RMD: Right Most Derivation

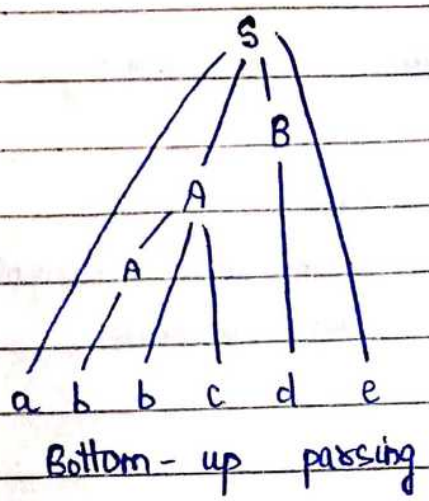
S → aA**B**e

S → a**A**de

S → a**A**bcde

S → abbcde

reverse



Scanned by CamScanner

2 3 4

Q. $a+b*c$

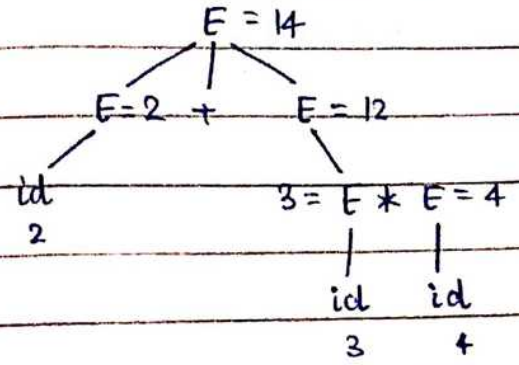
$E \rightarrow E+E \mid E*E \mid id$

Sol^n LMD: $E \rightarrow E+E$

$E \rightarrow a+E*E$

$E \rightarrow a+b*E$

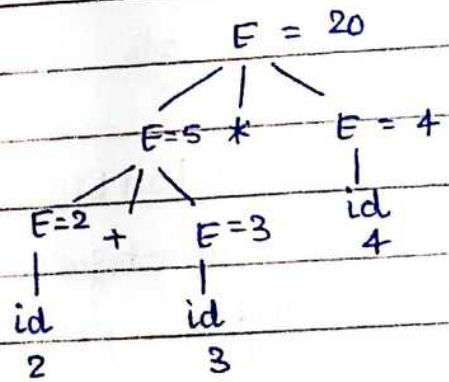
$E \rightarrow a+b*c$



LMD: $E \rightarrow E*E$

$E \rightarrow E+E*E$

$E \rightarrow a+b*c$



Q. $S \rightarrow aSbS \mid bSaS \mid \epsilon$ $\quad w = abab$

Sol^n $S \rightarrow aSbS$

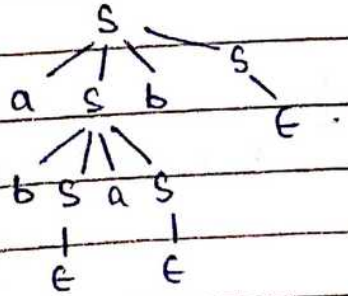
$S \rightarrow abSaSbS$

$S \rightarrow ab\epsilon aSbS$

$S \rightarrow aba\epsilon bS$

$S \rightarrow ababe$

$S \rightarrow abab$



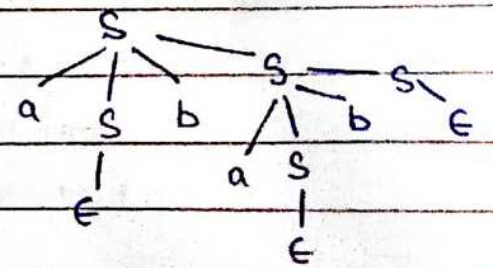
$S \rightarrow aSbS$

$S \rightarrow abS$

$S \rightarrow abaSbS$

$S \rightarrow ababS$

$S \rightarrow abab$



Scanned by CamScanner

Removing ambiguity

$E \rightarrow E+T/T$ left recursive
$T \rightarrow T*f/F$ left recursive
$f \rightarrow G \uparrow F/G$ right recursive
$G \rightarrow id$

Highest precedence gets lowest place is hierarchy.

Q. $R \rightarrow R+R/RR/R*/a/b/c$
Convert into unambiguous grammars.

Sol.ⁿ $R*$ ↓ Highest precedence
RR
$R+R$

~~$R \rightarrow R+R$~~
~~$R \rightarrow R+RR$~~

$E \rightarrow E+T/T$ → to make it left recursive
$T \rightarrow TF/f$
$f \rightarrow f*/a/b/c$

Q. bExp → bExp and bExp / bExp or bExp / not bExp / True / False
Convert into unambiguous order.

Sol.ⁿ $E \rightarrow E$ or T/T
$T \rightarrow T$ and f/f
$f \rightarrow$ not $f/$ True / False ↓ Highest Precedence

Q. $E \rightarrow E*f$ $*$ > $*$
$E \rightarrow f+E$ $+$ < $+$
$E \rightarrow f$ $+$ = $*$
$f \rightarrow f-f/id$ '$-$' has highest precedence.

Scanned by CamScanner

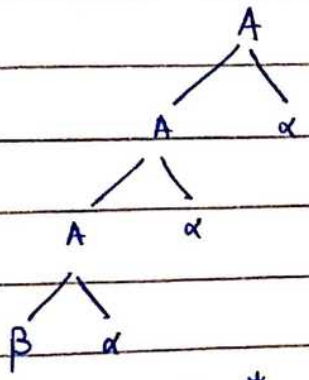
Recursion

Left

$A \rightarrow A\alpha / \beta$

```
A() {
    A()
    α
}
```

It may go into infinite loop.

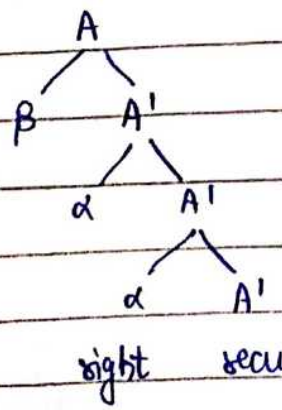


$\beta\alpha^*$

$A \rightarrow A\alpha / \beta$

or $A \rightarrow \beta A'$

$A' \rightarrow \epsilon / \alpha A'$



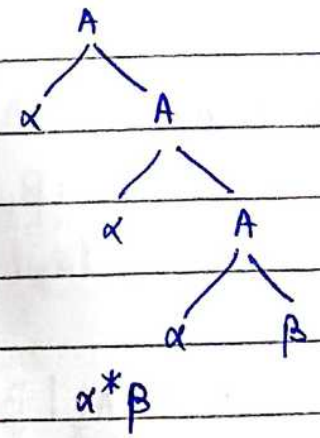
right recursive

Right

$A \rightarrow \alpha A / \beta$

```
A() {
    α
    A()
}
```

No infinite loop hence right recursion is not a problem



$\alpha^*\beta$

Q. $\underset{A}{E} \rightarrow \underset{A}{E} \underset{\alpha}{+} \underset{\beta}{T} / \underset{\beta}{T}$

Sol.n $E \rightarrow TE'$

$E' \rightarrow \epsilon / +TE'$

Left recursion has been removed.

Scanned by CamScanner

Q. $S \rightarrow (L) \mid x$

$L \rightarrow \underbrace{L}_{A} \underbrace{S}_{\alpha} \mid \underbrace{S}_{\beta}$ → left recursive

Sol.ⁿ

$L \rightarrow \beta L'$ or $L \rightarrow SL'$ (as $\beta = S$)

$L' \rightarrow \epsilon \mid ,SL'$

$S \rightarrow (L) \mid x$

Q. $\underbrace{S}_{A} \rightarrow \underbrace{SOS}_{A\ \alpha} \mid \underbrace{1}_{\beta}$

Sol.ⁿ $S \rightarrow 1S'$

$S' \rightarrow \epsilon \mid OS1SS'$

Q. $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \beta_1 \mid \beta_2 \mid \beta_3$

Sol.ⁿ $A \rightarrow AC \mid \beta_1 \mid \beta_2 \mid \beta_3$

$C \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$

$A \rightarrow \underbrace{AC}_{\alpha} \mid \underbrace{B}_{\beta}$

$B \rightarrow \beta_1 \mid \beta_2 \mid \beta_3$

$C \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$ ∴

$A \rightarrow BA'$

$A' \rightarrow \epsilon \mid CA'$

$C \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$

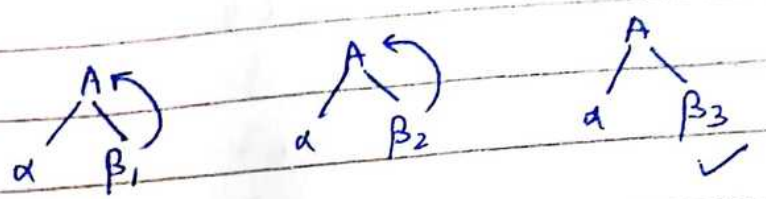
$B \rightarrow \beta_1 \mid \beta_2 \mid \beta_3$

Scanned by CamScanner

Non - Determinism

$A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \alpha\beta_3$

Given : $\alpha\beta_3$



$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 / \beta_2 / \beta_3$ (no backtracking required)

Q. $S \rightarrow iEtS / iEtSeS / a$

$E \rightarrow b$

Sol.n $S \rightarrow iETS / S' / a$

$S' \rightarrow \epsilon / eS / a$

$E \rightarrow b$

Q. $S \rightarrow aSSbS / aSaSb / abb / ab$

Sol.n $S \rightarrow aS' / b$

$S' \rightarrow SSbS / SaSb / bb$

Parsers

Q. $E \rightarrow E+T / T$

$T \rightarrow T*F / F$

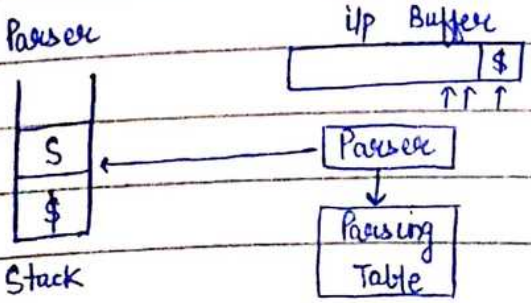
$F \rightarrow (E) / id$

Sol.n $A \rightarrow TA'$

$A' \rightarrow \epsilon / \alpha A'$

$\Rightarrow$

	first( )
$E \rightarrow TE'$	{(, id}
$E' \rightarrow \epsilon / +TE'$	{+, ε}
$T \rightarrow FT'$	{(, id}
$T' \rightarrow \epsilon / *FT'$	{*, ε}
$F \rightarrow (E) / id$	{(, id}



Scanned by CamScanner

		first()	follow()
Q.	S → ABCDE	{a, b, c}	{$}
	A → a|ϵ	{a, ϵ}	{b, c}
	B → b|ϵ	{b, ϵ}	{c}
	C → c	{c}	{e, $} {d, e, $}
	D → d|ϵ	{d, ϵ}	{e, $}
	E → e|ϵ	{e, ϵ}	{$}

Soln

		first()	follow
Q.	S → Bb|cd	{a, c, b, d}	{$}
	B ⇒ aB|ϵ	{a, ϵ}	{b}
	C → cC|ϵ	{c, ϵ}	{d}

		first()	follow()
Q.	E → TE'	{(, id}	{$,)}
	E' → +TE'|ϵ	{+, ϵ}	{$,)}
	T → FT'	{(, id}	{+, $,)}
	T' → *FT'|ϵ	{*, ϵ}	{+, $,)}
	F → (E)|id	{(, id}	{*, ϵ, +, $,)}

		first()	follow()
Q.	S → ACB|CbB|Ba	{d, h, b, a, g, ϵ}	{$, a, h}
	A → da|BC	{d, g, h, ϵ}	{h, g, $}
	B → g|ϵ	{g, ϵ}	{$, a, b, g}
	C → h|ϵ	{h, ϵ}	{g, $, b, h}

Scanned by CamScanner

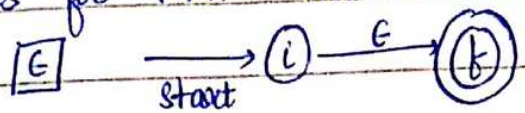
Q.1. $L = \{ (01)^i 1^{2j} \mid i \geq 1, j \geq 1 \}$

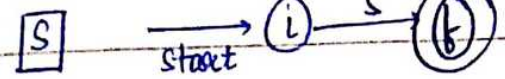
Q.2. $L = \{ w : n(a) = 1, w \in (a,b)^* \}$

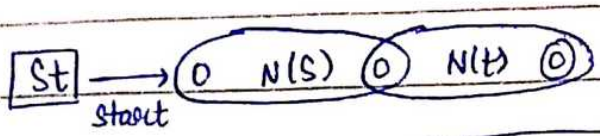
Q.3. $L = \{ w : n_a(w) \geq 1 ; w \in (a,b)^* \}$
↳ atleast 1 a

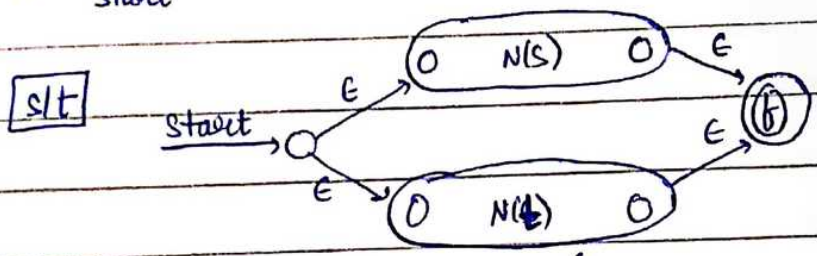
Q.4. $ab(c|d)^*c$
(NFA with ϵ)

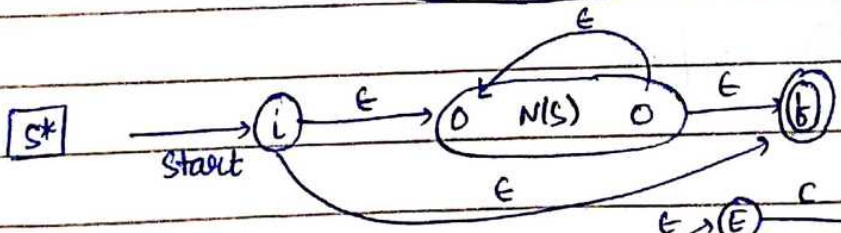
Rules for NFA with ϵ transitions :

1. 

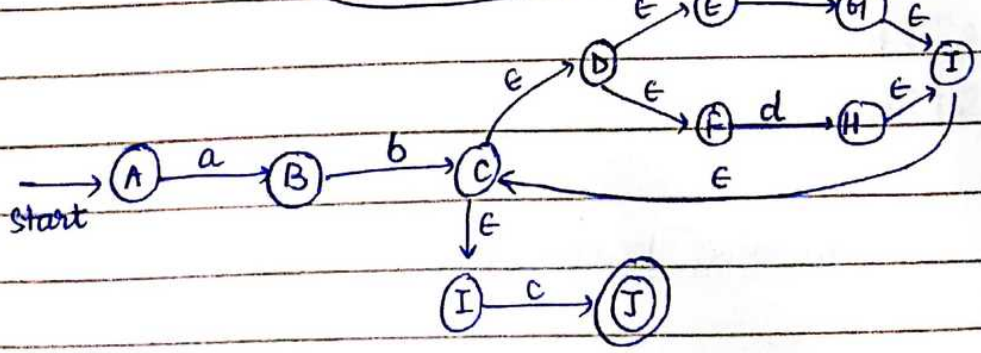
2. 

3. 

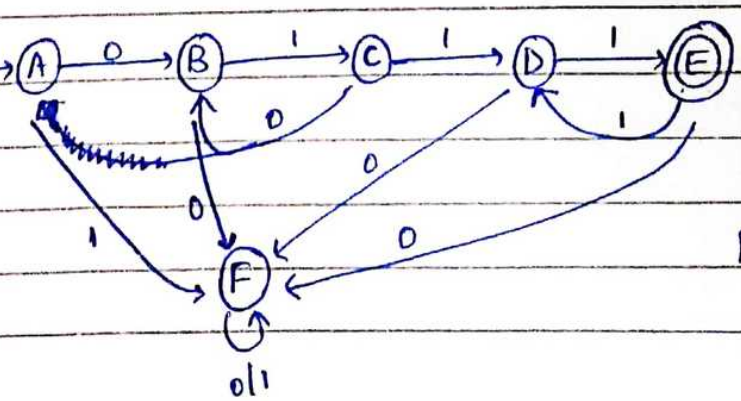




Sol.4.



Sol.1. DFA:

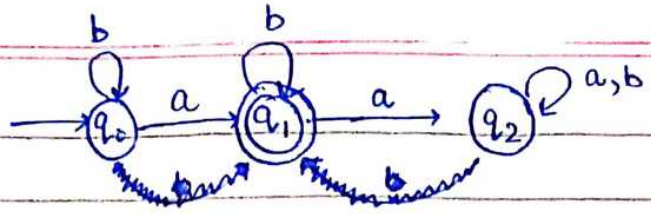


01010111

F: Dead State

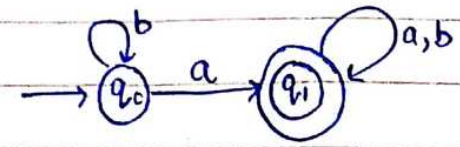
Scanned by CamScanner

Sol.2.



abbba
bbabb

Sol.3.



aabbb
bba

Q.5. Remove non-determinism. (common portion)

$S \rightarrow \underbrace{bSSaaS}_{\alpha\ \beta_1} / \underbrace{bSSaSb}_{\alpha\ \beta_2} / \underbrace{bSb}_{\alpha\ \beta_3} / a$

Sol.n

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 / \beta_2 / \beta_3$

$S \rightarrow bSS' / a$

$S' \rightarrow \underbrace{SaaS}_{\alpha\ \beta_1} / \underbrace{SaSb}_{\alpha\ \beta_2} / b$

$\Rightarrow S \rightarrow bSS' / a$

$S' \rightarrow SaS'' / b$

$S'' \rightarrow aS / Sb$

Recursive decent parsing

eg. $E \rightarrow E+i / i$

Removing left recursion: $E \rightarrow iE'$

$E' \rightarrow +iE' / \epsilon$

Scanned by CamScanner

i+i$

```
E()
{
    if (l == 'i')
    {
        match ('i');
        E'();
    }
}
```

```
E'()
{
    if (l == '+')
    {
        match ('+');
        match ('i');
        E'();
    }
    else return;
}
```

```
match (char t)
{
    if (l==t)
        l=getchar();
    else
        cout<<"EOF";
}
```

```
main(){
    E();
    if (l == '$')
        cout<<"Parsing successful";
}
```

1. Design a recursive decent parser for:

   Exp → digit rest

   rest → + digit rest | - digit rest | ϵ

   digit → 0|1|2|. . . |9

Scanned by CamScanner

Trailing (A):

1. If $A \to \alpha a \beta$ is a production where β is a single non-terminal or ϵ then add 'a' to trailing (A).
2. If $A \to \alpha B$ is a production then add trailing of (B) to trailing of (A).

	Leading	Trailing
$E \to E+T$ (α a β)	{ +, leading (T) }	{ +, trailing (T) }
$E \to T$	{ +, *, id, (}	{ +, *, id,) }
$T \to T*F$ (α a β)	{ *, leading (F) }	{ *, trailing (F) }
$T \to F$	{ *, id, (}	{ *, id,) }
$F \to id$ (α a β)	{ id, (}	{ id,) }
$F \to (E)$ (α a β)		

α a β → null

Operator Precedence Parsing Table

1. $\$ < \theta$ where θ : All terminals present in the leading (A) and $\theta > \$$ for all terminals present in trailing (A).
2. For each production $A \to X_1 X_2 \cdots X_n$ do
   for $i = 1$ to $n-1$ do
   begin
   1. if X_i and X_{i+1} are both terminals then set $X_i = X_{i+1}$
   2. if $i \le n-2$ and X_i and X_{i+2} are terminals and X_{i+1} is a nonterminal then set $X_i = X_{i+2}$

Scanned by CamScanner

3. if X_i is a terminal and X_{i+1} is a nonterminal then for all a in leading (X_{i+1}) do set $X_i < a$

4. if X_i is a non terminal and X_{i+1} is a terminal then for all a in Trailing (X_i) do set $a > X_{i+1}$

	leading	trailing
$E \to E+T$	{+, *, id, (}	{+, *, id,)}
$E \to T$		
$T \to T*F$	{*, id, (}	{*, id,)}
$T \to F$		
$F \to id$	{id, (}	{id,)}
$F \to (E)$		

	+	*	(	)	id	$
+	>	<	<	>	<	>
*	>	>	<	>	<	>
(	<	<	<	=	<	
)	>	>		>		>
id	>	>		>		>
$	<	<	<		<	

Scanned by CamScanner

L-R Parsers (Shift Reduce Parsers)

→ Shift (Push)
→ Reduce (Pop)
→ Accept
→ Error

Q $S \to aAcBe$ $\quad w = abbbbcde$

$A \to Ab/b$

$B \to d$ $\quad$ Use RMD.

Sol^n $S \to aAcBe$

$S \to aAcde$

$S \to aAbcde$

$S \to aAbbcde$

$S \to abAbbbcde$

$S \to abbbbcde$

A handle of a right sentinel form γ is a production of the form $A \to B$ and a position of γ where the string β may be found and replaced by A to produce the previous right sentinel form in a rightmost derivation of γ.

Scanned by CamScanner

Q Design LL(1) parsing table :

$A \to \underbrace{AcB}_{\alpha} \mid \underbrace{cC}_{\beta} \mid C$

$B \to bB \mid id$

$C \to \underbrace{CaB}_{\alpha} \mid \underbrace{BbB}_{\beta} \mid B$

Sol"

	first()	follow()
$A \to cCA' \mid CA'$	{c, b, id}	{b, a, id, $, c}
$A' \to \epsilon \mid cBA'$	{ε, c}	{b, id, a, $, c}
$B \to bB \mid id$	{b, id}	{b, id}
$C \to BbBC' \mid BC'$	{b, id}	{b, a}
$C' \to \epsilon \mid aBC'$	{ε, a}	{b, id, a}

BC'A' : id a A'

BbBC' : id b BC'

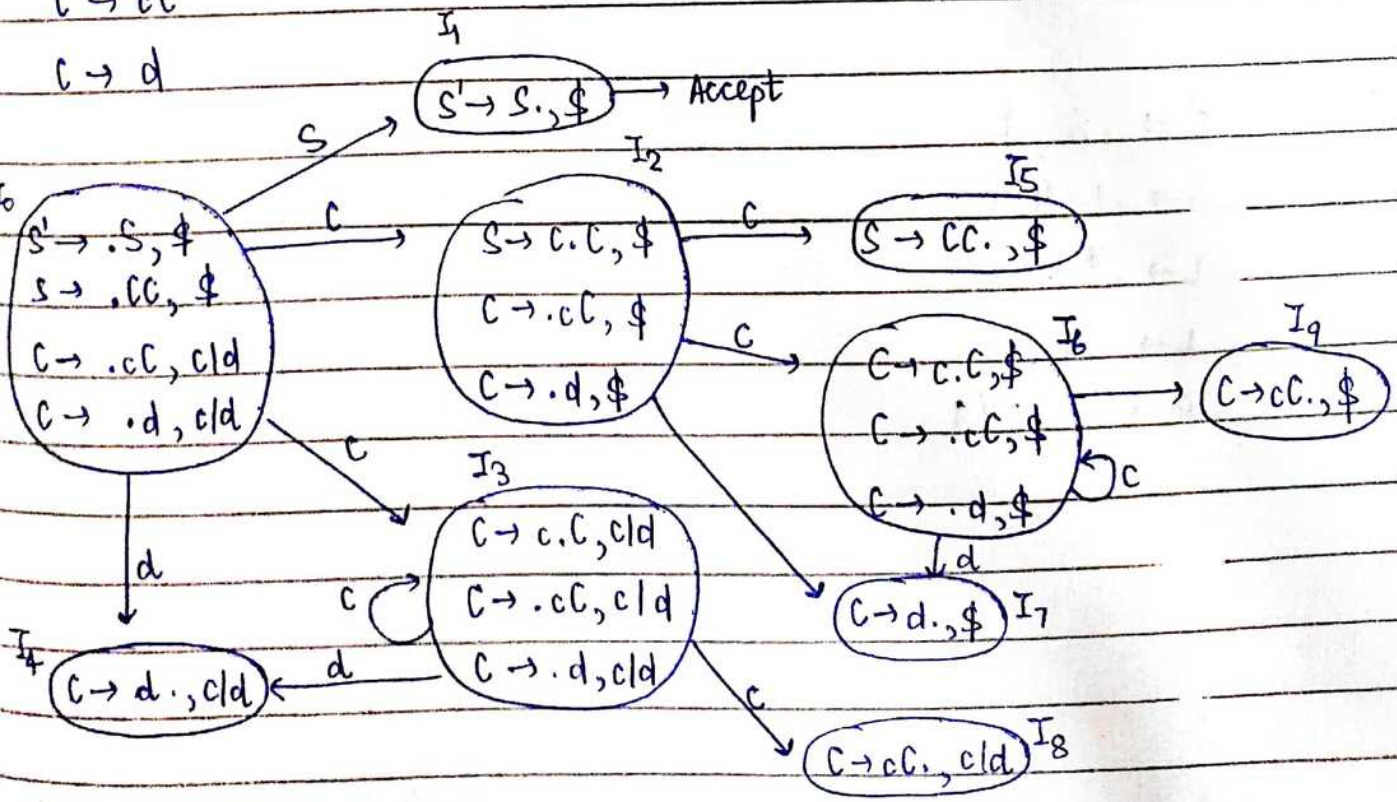
CLR(1) Parsing Table

$S' \to S$

$S \to CC$

$C \to cC$

$C \to d$



Scanned by CamScanner

LALR
SLR
CLR

State	c	d	$	S	C
0	S_3	S_4		1	2
1			Accept		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

Q. $S \to L=R$ Construct CLR(1) parsing table.

$S \to R$

$L \to *R$

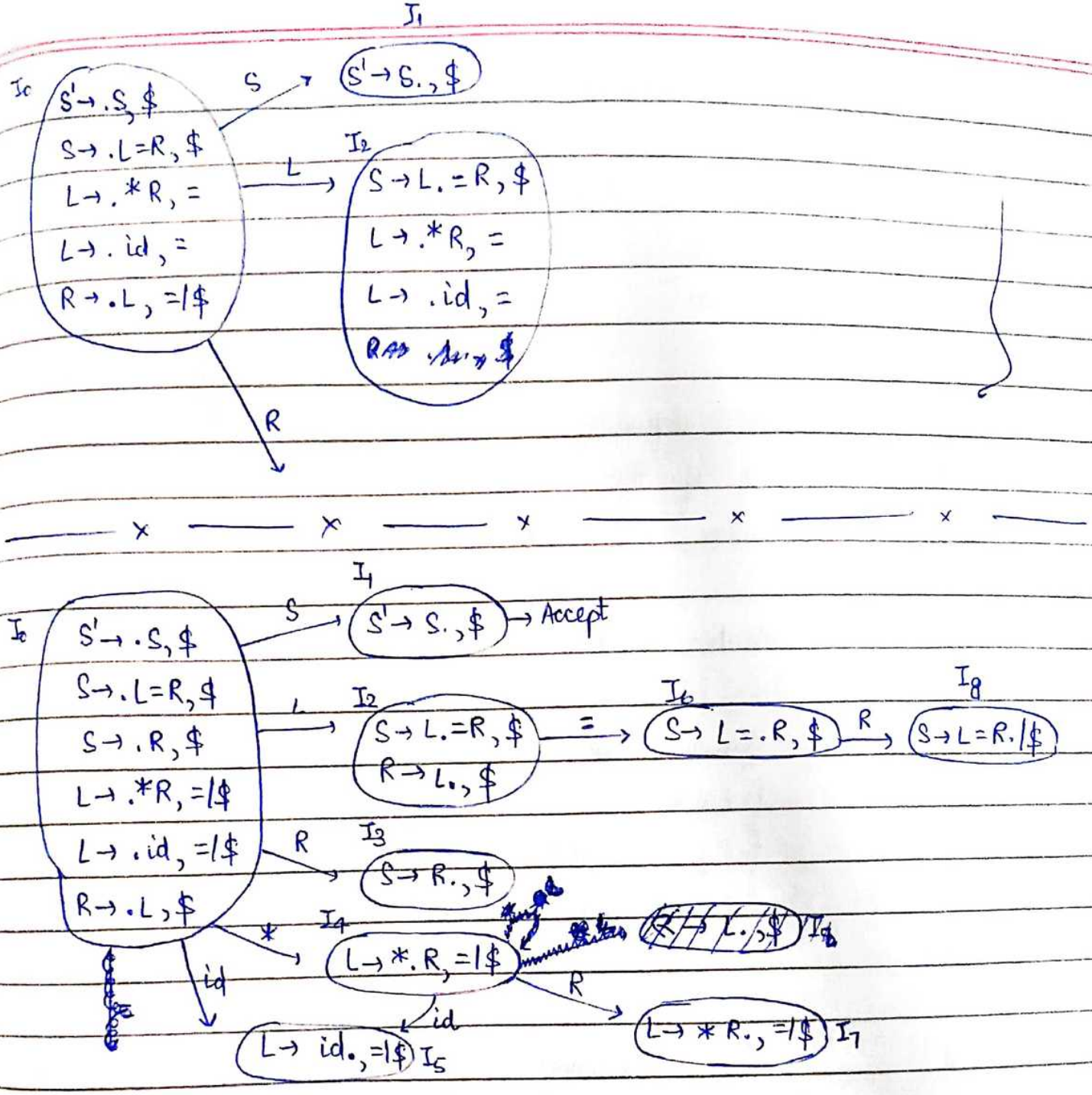
$L \to id$

$R \to L$

Sol.n

$S' \to .S, \$$	$S' \to .S, \$$
$S \to .L=R, \$$	$S \to .L=R, \$$
$L \to .*R, =$	$S \to .R, \$$
$L \to .id, =$	$L \to .*R, =/\$$
$R \to .L, =/\$$	$L \to .id, =/\$$
	$R \to .L, \$$

Scanned by CamScanner



State	=	*	id	$	S	L	R
0							
1							
2							
3							
4							
5							
6							
7							
8							
9							

Scanned by CamScanner

Syntax Directed Translation

α + Semantic Rules / => SDT

Grammar Action

Production

SDD: Syntax directed definition

Hides the complexities of the program

No order of evaluation is considered.

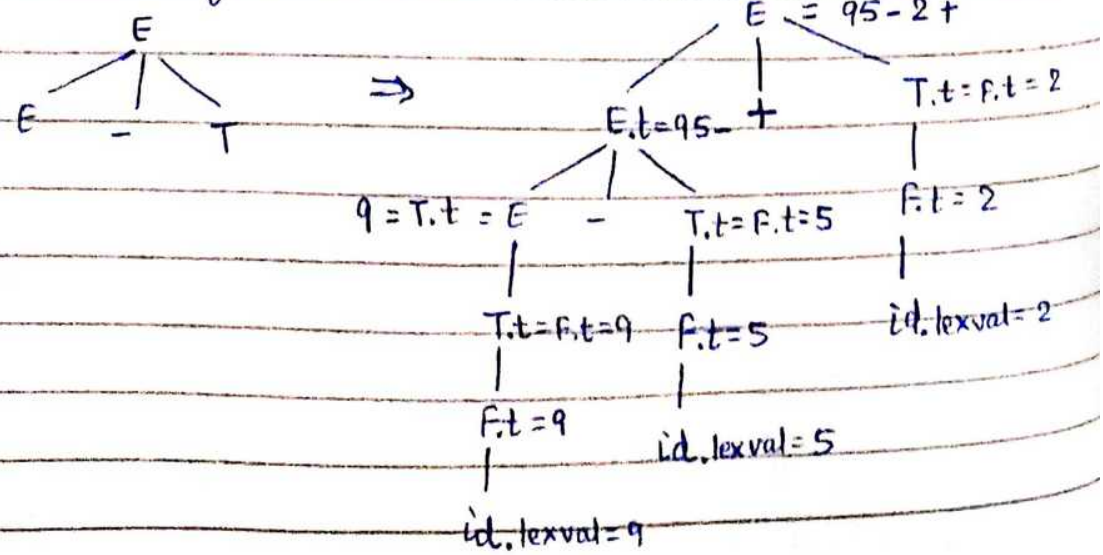
SDT: Order of evaluation matters.

Q.

SDD:	Semantic Rule:
$E \to E+T$	$\{ E.t = E_{1}.t \,\|\|\, T.t \,\|\, '+' \}$
$E \to E-T$	$\{ E.t = E_{1}.t \,\|\| T.t \,\|\| - \}$
$E \to T$	$\{ E.t = T.t \}$
$T \to T*F$	$\{ T.t = T_{1}.t \,\|\| F.t \,\|\| '*' \}$
$T \to F$	$\{ T.t = F.t \}$
$F \to id$	$\{ F.t = id.lexval \}$

Generate parse tree for 9-5+2.

Soln

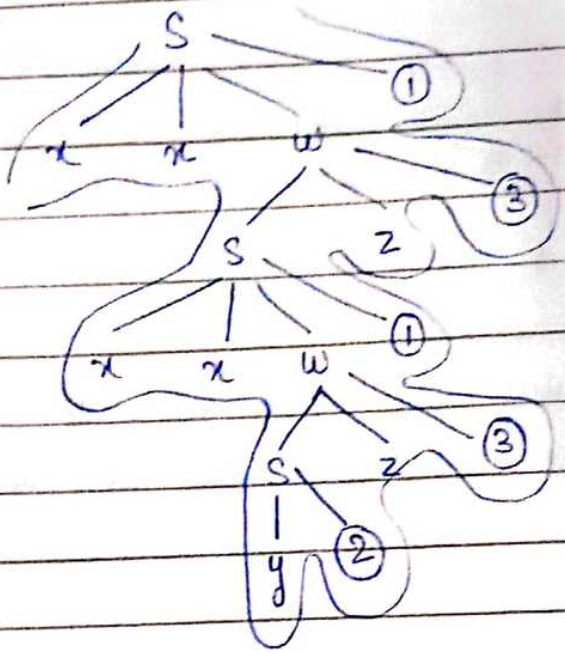


Scanned by CamScanner

6. ① $S \to xxw$ { Print 6 }
③ $w \to sz$ { Print 2 }
② $s \to y$ { Print 5 }

String: xxxxyzz

Sol^n

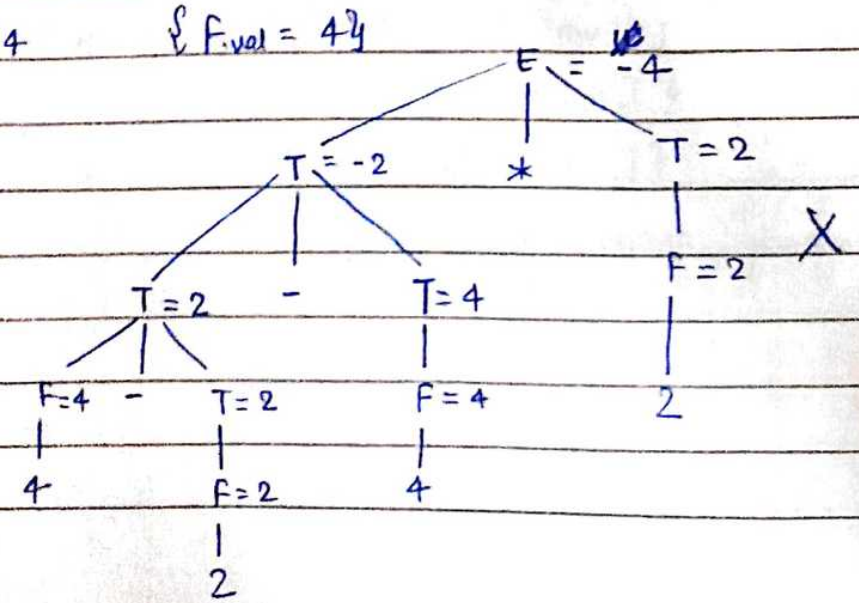


~~5 6 1 5 2~~

52626

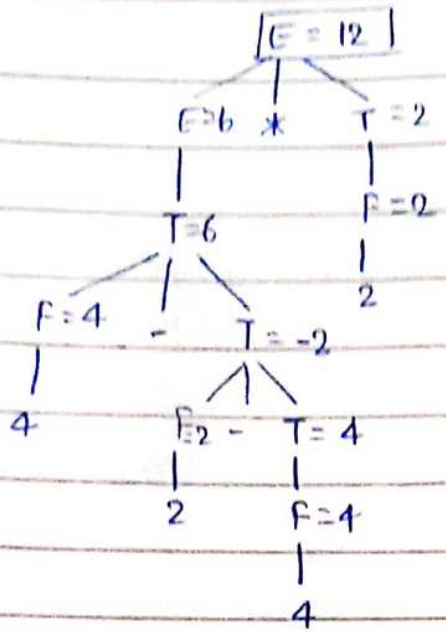
8. ① $E \to E * T$ { E.val = E1.val * T.val }
② / T { E.val = T.val }
③ $T \to F - T$ { T.val = F.val T1.val }
④ / F { T.val = F.val }
⑤ $F \to 2$ { F.val = 2 }
⑥ / 4 { F.val = 4 }

$w = 4 - 2 - 4 * 2$



X

Scanned by CamScanner



Total Reductions = 10

Synthesized Attribute: The value of synthesized attrs. at a node is computed from the value of the attrs. at the children of that node.

ASSIGNMENT 2:

1. Synthesized & inherited attribute
2. S-attributed & L-attributed grammar
   - with eg.
3. How to draw dependency graph from SDT?
4. Given grammar is:

$T \rightarrow FT'$	$\{ T.inh = F_1.val \; ; \; T.val = T'.syn \}$
$T' \rightarrow *FT'$	$\{ T'_1.inh = T'.inh * F.val \; ; \; T'.syn = T'_1.syn \}$
$T' \rightarrow \epsilon$	$\{ T'.syn = T'.inh \}$
$F \rightarrow digit$	$\{ F.val = digit.lexval \}$

Draw the dependency graph for 3*5.

Scanned by CamScanner

Representation of Intermediate code

1. Syntax Tree (Abstract Parse Tree)
2. DAG (Directed Acyclic Graph)
3. Postfix
4. Three address code → Quadruples
   → Triple
   → Indirect triple

eg. $x = (a * b) + (a * b)$

$S \rightarrow i = E$

$E \rightarrow E+E \mid E*E \mid (E) \mid id \mid E-E$

Construct the parse tree.

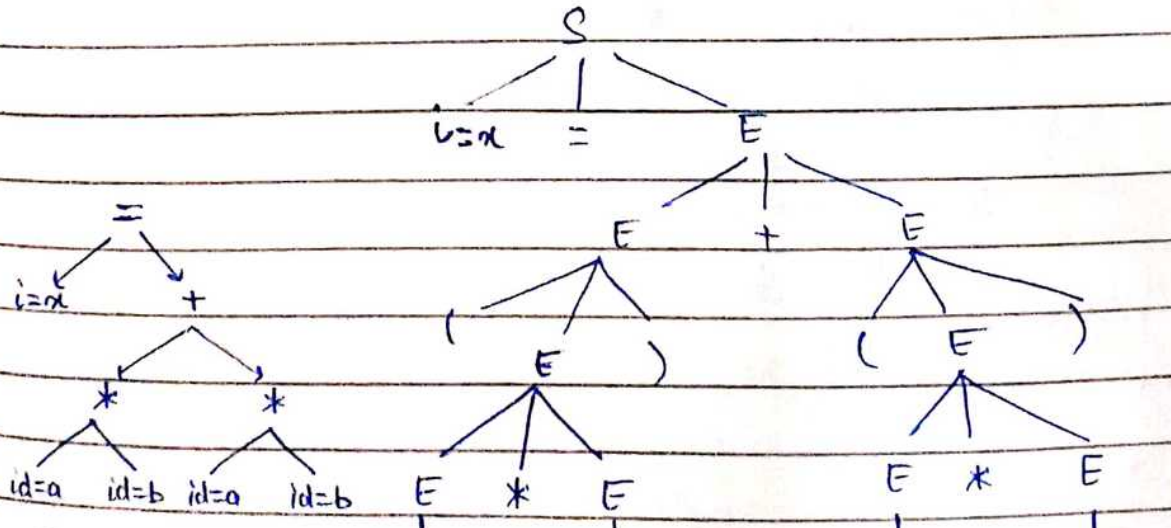
Sol.n

$S \rightarrow i = E$

$\rightarrow i = E+E$

$\rightarrow i = (E*E) + (E*E)$

$\rightarrow i = (id * id) + (id * id)$

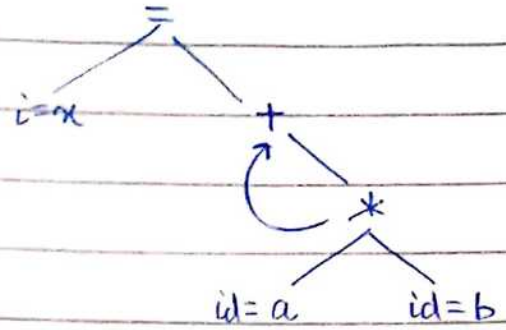


SYNTAX TREE
(only the terminals)

CONCRETE PARSE TREE

Scanned by CamScanner

DAG:



Construct the DAG for the given expression : $a + a * (b-c) + (b-c) * d$

$x = a + a * (b-c) + (b-c) * d$

$S \rightarrow i = E$

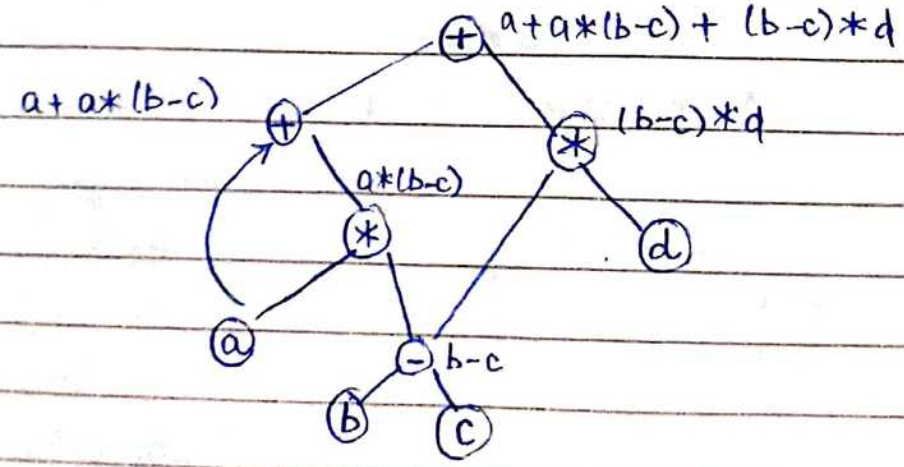
$\rightarrow i = E + E$

$\rightarrow i = id + E + E$

$\rightarrow i = id + E * E + E * E$

$\rightarrow i = id + id * (E-E) + (E-E) * id$

$\rightarrow i = id + id * (id-id) + (id-id) * id$



DAG

Scanned by CamScanner

Q. Convert to postfix : (i) If C then A else B.

Sol^n CAB ?

(iii) if C then (if P then q else R) else B

Sol^n CPQR? B?

* Quadruple (operator, arg1, arg2, Result) :

eg. $A = -B * (C + D)$

$t_1 = -B$

$t_2 = C + D$

$t_3 = t_1 * t_2$

$A = t_3$

	Operator	arg1	arg2	result
(0)	uminus	B	-	t_1
(1)	+	C	D	t_2
(2)	*	t_1	t_2	t_3
(3)	=	t_3	-	A

* Triple (using pointers) :

	op.	arg1	arg2
(0)	uminus	B	-
(1)	+	C	D
(2)	*	(0)	(1)
(3)	=	(2)	-

Reordering becomes difficult due to pointers.

* Indirect Triple :

	Statement
(0)	14
(1)	15
(2)	16
(3)	17

Scanned by CamScanner

Q. Generate the 3-address code: $x = (b*d) + a - b$

Sol.ⁿ

$t_0 = b*d$

$t_1 = t_0 + a$

$t_2 = t_1 - b$

$A = t_2$

(0)	*	b	d	t_0
(1)	+	t_0	a	t_1
(2)	-	t_1	b	t_2
(3)	=	t_2	-	A

(+ and - are left associative.)

Q. Give the 3-address code: If (A < B) then 1 else 0.

Sol.ⁿ

1. If (A<B) then goto (4)
2. T = 0
3. goto (5)
4. T = 1
5. next

Q.
```
while (s<d)
{
    if (x<y)
        z = a+b*c
    else
        z=0
}
```

Sol.ⁿ

Scanned by CamScanner